

Informe técnico - Pipeline ML End-to-End con MLOps

Tarea 2: Implementación de Pipeline ML End-to-End

Autor: Julio Rios

1. Resumen ejecutivo

El proyecto implementa un pipeline completo de Machine Learning orientado a resolver un caso de priorización analítica de clientes. El flujo automatiza las etapas de ingesta, preprocesamiento, entrenamiento, evaluación, registro del modelo, monitoreo, postprocesamiento, generación de archivos de réplica y exposición del modelo mediante una API de inferencia. La solución aplica principios de MLOps para asegurar reproducibilidad, trazabilidad y preparación para ambientes productivos.

2. Objetivo del pipeline

El objetivo es transformar datos históricos en una herramienta de decisión que permita priorizar acciones de negocio. El modelo estima una probabilidad de ocurrencia del evento objetivo y, posteriormente, combina dicha probabilidad con variables de valor y frescura para construir una puntuación TLV. Esta puntuación permite ordenar los clientes y asignarlos a grupos de ejecución.

3. Arquitectura del proyecto

Componente	Propósito
main.py	Orquesta la ejecución completa del pipeline.
preprocessing.py	Limpia datos, trata nulos, codifica variables y genera train/test/validation.
training.py	Entrena XGBoost, optimiza hiperparámetros con Optuna y registra en MLflow.
monitoring.py	Calcula PSI, AUC, recall y genera archivos de monitoreo.
postprocessing.py	Calcula TLV, segmenta clientes y genera archivos de réplica.
api.py	Expone el modelo mediante FastAPI.
dashboard.py	Visualiza resultados y métricas mediante Streamlit.

4. Flujo End-to-End

El pipeline se ejecuta desde main.py y sigue una secuencia reproducible: 1) lectura del dataset; 2) limpieza y transformación; 3) partición temporal y aleatoria; 4) entrenamiento con búsqueda de hiperparámetros; 5) registro de métricas y modelo en MLflow; 6) cálculo de métricas de monitoreo; 7) postprocesamiento de scores; 8) generación de archivos pipe-delimitados para réplica; 9) consumo del modelo por API; y 10) visualización en dashboard.

5. Trazabilidad y versionado

El proyecto registra los experimentos en MLflow, incluyendo parámetros, métricas y artefactos del modelo. Además, guarda una versión local del modelo en models/model.pkl y la lista de variables utilizadas en models/feature_names.json. La metadata del preprocesamiento se conserva en data/processed/preprocessing_metadata.json, lo cual permite mantener consistencia entre entrenamiento e inferencia.

6. Evaluación y monitoreo

Métrica	Uso
AUC	Mide la capacidad del modelo para discriminar entre clases.
Recall	Permite evaluar la capacidad de capturar casos positivos.
Recall por decil	Evalúa la concentración de casos positivos en los grupos de mayor score.
PSI	Detecta deriva entre la distribución de scores de entrenamiento y validación.

Los umbrales aplicados para PSI son: menor a 0.10 = OK, entre 0.10 y 0.25 = advertencia, y mayor a 0.25 = alerta de deriva severa.

7. API de inferencia

La API se implementa con FastAPI y permite ejecutar inferencias individuales o por lote. Los endpoints principales son /health, /predict y /predict-batch. La documentación interactiva queda disponible en /docs al ejecutar `uvicorn api:app --reload`. La API carga el modelo entrenado y alinea automáticamente las variables recibidas con la estructura utilizada durante el entrenamiento.

8. Dashboard

El dashboard se implementa con Streamlit y permite revisar la distribución de grupos TLV, los clientes con mayor puntuación, las métricas de monitoreo, el recall acumulado por decil y la distribución de scores. Esta vista facilita la comunicación de resultados a usuarios técnicos y de negocio.

9. Valor de negocio

La solución permite priorizar clientes según probabilidad estimada y valor esperado, reduciendo el esfuerzo manual de análisis y enfocando los recursos en los segmentos con mayor impacto. Para la organización, esto se traduce en mejor eficiencia operativa, mayor trazabilidad de decisiones, reducción de tiempos de análisis, mejor gobierno del modelo y capacidad de escalar la solución hacia un entorno productivo.

10. Reproducibilidad y ejecución

La reproducibilidad se garantiza mediante una estructura modular, archivo requirements.txt, semilla de entrenamiento, salidas estandarizadas y documentación en README.md. La ejecución completa se realiza con `python main.py`. Luego, los experimentos se revisan con mlflow ui, la API se inicia con `uvicorn api:app --reload` y el dashboard con `streamlit run dashboard.py`.

11. Conclusiones

El pipeline desarrollado cumple con los requerimientos de una solución ML End-to-End: automatización, versionado, monitoreo, inferencia y comunicación ejecutiva del impacto. Además, establece una base técnica ordenada para evolucionar hacia despliegues productivos con mayor control de datos, seguridad, automatización CI/CD y monitoreo continuo.